**Gantry: Your Plan as Pseudocode**

This is your step-by-step plan (part a) written as pseudocode, with each section labeled by its owner from your pieces table (part b). Your plan is the most precise in the class: almost every line can only be coded one way. Only three places are marked **???** in red.

---

**The goal:** a developer who has never met your team could code this algorithm and get exactly your expected outputs. Every ??? is a place where that developer would have to guess.

---

**Algorithm 1** One Safe Move (your steps)

**Input:** grid size; blocked cells; homed (yes/no); target (x, y)
**Output:** the route from (0, 0) to the target, or an error
*Workspace map — Lance; input validator — Jay*
load the grid and the blocked cells; start ← (0, 0)
read the target (x, y)
**if** the target is outside the grid **then** print "Error: target out of bounds."; **stop**
**if** the target is blocked **then** print "Error: target blocked."; **stop**
*Controller — Ricardo*
**if** not homed **then** print "Error: not homed."; **stop**
*Pathfinder — Tex*
open ← {start}; g(start) ← 0
**while** open is not empty **do**
    c ← the cell in open with the lowest f = g + h (g = steps from the start, h = |dx| + |dy| to the target); **if** two or more cells tie on f **then** pick the one with the lower h; **if** they still tie **then** pick the one that was added to open first (neighbors are added in the order right (x+1, y), up (x, y+1), left (x−1, y), down (x, y−1); a cell whose g is updated while in open keeps its original place in that order; a reopened cell counts as newly added)

```
11:         if c = target then build the
            route by following parents back;
            leave the loop
12:         move c from open to closed
13:         for each neighbor n of c (right,
            up, left, down), inside the grid
            and not blocked do
14:         if g(c) + 1 ≥ g(n) then skip it (a
            cell not yet seen has g(n) = ∞)
15:         g(n) ← g(c) + 1; parent(n) ← c;
            if n is in closed then move n
            from closed back to open (reopen
            it); else if n is not in open then
            add n to open
16:         end for
17:     end while
18:     if the target was not reached then print
        "Error: no route found."; stop
Main and tests — Imprezz
19:     print "Route: " and every cell of the route in
        order
```

**A note on your plan**

- Very clear work: exact steps, exact messages, and a stated coordinate rule (x to the right, y up).
- Your plan says "A* search," which is a good choice. Every person on the team should be able to explain line 10 (lowest f, and why h is the Manhattan distance).
- Your owners are "proposed." Confirm them today.

### Your path

Each sprint ends with something you can see working.

| Sprint | Dates | Goal |
|---|---|---|
| **1** | now → Oct 29 | **Your first CORE item.** A safe route on a grid with obstacles. |
| **2** | Oct 29 → Nov 12 | **Your second CORE item.** Safety: homing, the stop button, and the load limit. |

### What a precise line looks like

A line is precise when two developers would code it the same way. Vague words like "pick," "try another," "handle," or "use other info" leave the decision open.

A precise line is a comparison, a sort, or a calculation. An example from the trash can:

| Vague (two people code it differently) | Precise (everyone codes it the same) |
|---|---|
| check the reading | **if** reading $< 0$ **or** reading $> 100$ **then** ignore it |
| show the fullest bins first | sort the bins by percent full, highest first |
| mark full bins | **if** percent $> 90$ **then** mark "Full" |

*Your ??? lines are where your algorithm makes its decisions. Each one should become a line like the ones on the right.*

**What each ??? means**

Each ??? is a place where two people on your team could code it differently, and both would think they followed the plan. For you, they matter because your test cases print one exact route.

| ??? | The question | Why it is a problem (example) |
|---|---|---|
| **1** | Neighbor order of what? | Two cells in open can have the same f and h but come from different cells. "Right, up, left, down" describes the neighbors of one cell, not how to choose between cells from different parents. If the tie-break differs, the printed route differs, and an exact test fails. |
| **2** | Skip, or reopen? | Your step 4 says "if a shorter path to an open cell is found, update it," but it doesn't say what happens to a cell that was already expanded. With your heuristic, skipping is safe. Say so, so everyone codes it the same. |
| **3** | Why after the search? | Your step 6 checks homing after the route is found. If the gantry isn't homed, it doesn't know where (0, 0) is, so the search is meaningless. Check it before anything else. |

**Check your algorithm with your code**

Your plan is precise enough to code today. Use the code to check the algorithm: make your pathfinder print what it does in every round, and compare it with what you expect.

**Your test 2 from part c:** $3 \times 3$ grid, (1, 0) blocked, homed, target (2, 0). Expected: Route (0, 0) $\rightarrow$ (0, 1) $\rightarrow$ (1, 1) $\rightarrow$ (2, 1) $\rightarrow$ (2, 0). Here are the first two rounds of the search, worked by hand. Each cell in open is written as cell: g, h, f.

| Round | Expand c | Open after this round | Notes |
|---|---|---|---|
| 1 | (0, 0) | (0, 1): 1, 3, 4 | Right (1, 0) is blocked; left and down are outside. |
| 2 | (0, 1) | (1, 1): 2, 2, 4; (0, 2): 2, 4, 6 | Down (0, 0) is closed: skipped. |

*Your program's printout should show exactly these two rounds. If it doesn't, the code or the pseudocode is wrong.*

**Your turn (in class today)**

**Part 1. Fix the algorithm.** On page 1, replace the three red **???** with precise lines. Highlight the lines you changed.

**Part 2. Code it with a round-by-round printout.** Each owner codes their section. The pathfinder prints, for every round, the cell it expands and the open list with g, h, f. Run test 2: the first two rounds must match the table above, and the route must match your expected output.

**Part 3. Ties.** New test 6: 3 × 3 grid, nothing blocked, homed, target (2, 2). There are several shortest routes. Before running, write down which one your tie rule from Part 1 should print. Then run it. Do they match?

**Part 4. Run all six tests.** Your five tests from part c plus test 6. Each should print PASS or FAIL.

**Done early?** Start your Sprint 2 (see page 1). Write the new lines in your pseudocode first.

**Submit: one per team, by Wednesday, October 21, 9:00 AM.**
- Part 1: this document, page 1 with no ??? left, changed lines highlighted.
- Part 2: your program's printout for test 2 (all rounds and the route).
- Parts 3 and 4: your predicted route for test 6, and the PASS/FAIL output for all six tests.